

Applied Cloud Computing

Assignment 5 Simple E-Commerce Microservices

(updated Oct. 2025)

Project Deliverable:

1, After you test everything are working, bring down all containers then start them again by **\$docker compose up -d** cmd. Take a screenshot of this cmd and its outputs.

Ans -

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % docker compose up -d
[+] Running 24/24
  ✓ order-db Pulled                                17.5s
  ✓ a16e55119267 Pull complete                     9.1s
  ✓ 115691a23fdc Pull complete                     7.7s
  ✓ 350dcaafd261 Pull complete                     14.2s
  ✓ 5c2e25ca380b Pull complete                     7.2s
  ✓ 1130f2e13fb8 Pull complete                     7.2s
  ✓ bdab963b189e Pull complete                     7.2s
  ✓ cc88b5d238d8 Pull complete                     9.3s
  ✓ 6b0d17933b3b Pull complete                     9.5s
  ✓ f34d00eff0dc Pull complete                     7.2s
  ✓ b77acb7ca7f9 Pull complete                     7.2s
  ✓ 072c7f69ec80 Pull complete                     7.2s
  ✓ 059a685ce897 Pull complete                     6.6s
  ✓ 0d013ab68057 Pull complete                     7.1s
  ✓ product-db Pulled                             22.3s
  ✓ 9f0164b2cc35 Pull complete                     7.1s
  ✓ fb403185da21 Pull complete                     7.2s
  ✓ 14bdb55a4ac2 Pull complete                     9.2s
  ✓ fc8964700298 Pull complete                     7.2s
  ✓ 9eeee0fb9876 Pull complete                     7.2s
  ✓ f19012efbdb2 Pull complete                     7.2s
[+] Running 10/10
  ✓ assignment_microservices-product-service      Built                                0.0s
  ✓ assignment_microservices-user-service         Built                                0.0s
  ✓ assignment_microservices-order-service        Built                                0.0s
  ✓ Network assignment_microservices_default      Created                              0.0s
  ✓ Container assignment_microservices-product-db-1 Started                     0.7s
  ✓ Container assignment_microservices-order-db-1 Healthy                     6.2s
  ✓ Container assignment_microservices-user-db-1   Healthy                     6.2s
  ✓ Container assignment_microservices-product-service-1 Started                  0.7s
  ✓ Container assignment_microservices-user-service-1 Started                  6.1s
  ✓ Container assignment_microservices-order-service-1 Started                  6.1s
```

2, After all services are up, Take a screenshot of **\$docker ps** output which show all the running containers (6 containers)

Ans -

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
3b5ae138d35b   assignment_microservices-order-service "python app.py"         52 seconds ago Up 45 seconds 0.0.0.0:5003->5003/tcp, [::]:5003->5003/tcp assignment_microservices-order-service-1
b26f279b88ac   assignment_microservices-user-service "python app.py"         52 seconds ago Up 45 seconds 0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp assignment_microservices-user-service-1
fde2cb62e966   assignment_microservices-product-service "docker-entrypoint.s..." 52 seconds ago Up 51 seconds 0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp assignment_microservices-product-service-1
d028c83f06f1   postgres                                "docker-entrypoint.s..." 52 seconds ago Up 51 seconds (healthy) 5432/tcp assignment_microservices-user-db-1
9ea2f9acc50fb   postgres                                "docker-entrypoint.s..." 52 seconds ago Up 51 seconds (healthy) 5432/tcp assignment_microservices-order-db-1
da09adcbcd1    mongo                                  "docker-entrypoint.s..." 52 seconds ago Up 51 seconds      27017/tcp assignment_microservices-product-db-1
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices %
```

3, Screenshot of adding user (name and email), then retrieve the user information:

Ans -

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % curl -X POST http://localhost:5001/users -H "Content-Type: application/json" -d '{"name": "John Doe", "email": "john@example.com"}'
{"message": "User added!"}
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % curl -X GET http://localhost:5001/users
[[{"name": "John Doe", "email": "john@example.com"}]]
```

4, Screenshot of adding product, then retrieve the product information:

Ans -

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % curl -X POST http://localhost:5002/products -H "Content-Type: application/json" -d '{"name": "Laptop", "price": 1200}'
{"message": "Product added!"}
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % curl -X GET http://localhost:5002/products [{"_id": "6900308ad3365ca691031a9f", "name": "Laptop", "price": 1200, "__v": 0}]
```

5, Screenshot of adding order, then retrieve the order information:

Ans -

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % curl -X POST http://localhost:5003/orders -H "Content-Type: application/json" -d '{"user_id": 1, "product": "Laptop", "quantity": 3}'
{"message": "Order placed!"}
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % curl -X GET http://localhost:5003/orders [{"_id": "6900308ad3365ca691031a9f", "name": "Laptop", "price": 1200, "__v": 0}]
```

6, Show the log of user-service via using `$ docker logs your_user_service_container_name`

Ans -

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % docker logs assignment_microservices-user-service-1
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://172.18.0.7:5001
Press CTRL+C to quit
```

7, There are two postgres database had been created, find out the two database's name via `$ docker inspect postgre_container_ID | grep POSTGRES_DB`

Screenshot needs to show two db's name:

Ans -

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % docker inspect d028c83f06f1 | grep POSTGRES_DB
"POSTGRES_DB=users_db",
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % docker inspect 9ea2f9ac50fb | grep POSTGRES_DB
"POSTGRES_DB=orders_db",
```

8, Enter one of the postgres db(the users_db), via

`$ docker exec -it your_user_db_containerID psql -U postgres -d users_db`

Inside the postgres db, issue

>SELECT * FROM users;

Screenshot the result: (use **>exit** to exit the postgres db prompt)

Ans -

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % docker exec -it assignment_microservices-user-db-1 psql -U postgres -d users_db

psql (18.0 (Debian 18.0-1.pgdg13+3))
Type "help" for help.

users_db=# SELECT * FROM users;
 id | name | email
-----+-----+-----
  1 | John Doe | john@example.com
(1 row)
```

9, Do the similar operation on the 2nd postgres db (the **orders_db**). Screenshot:

Ans -

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % docker exec -it assignment_microservices-or
der-db-1 psql -U postgres -d orders_db
psql (18.0 (Debian 18.0-1.pgdg13+3))
Type "help" for help.

orders_db=# SELECT * FROM orders;
 id | user_id | product | quantity 
-----+-----+-----+-----
  1 |      1 | Laptop  |         3 
(1 row)
```

10, Enter the MongoDB shell via

\$ **docker exec -it your_mongoDB_containerID mongosh** , inside mongoddb shell, issue

>**show databases**

You should see few database names, like admin, config, local and **products** (product is the db we created). Issue

>**use products**

>**db.products.find().pretty()** (note: if the process does not show return message for long time, you need to hit 'enter' to force the message get displayed).

- Screenshot the output:

Ans -

```
[test> use products
switched to db products
[products> db.products.find().pretty()
[
  {
    _id: ObjectId('6900367bfb80b74c5d5e14c6'),
    name: 'Laptop',
    price: 1200,
    __v: 0
  }
]
products> █
```

- What this command 'db.products.find().pretty()' means ?

Ans -

db.products.find().pretty() is a MongoDB shell command used to retrieve all documents from the products collection in the current database and display them in a readable, formatted way. Here, db refers to the current database, products is the collection (similar to a table in SQL), find() fetches all documents in that collection, and pretty() formats the output with indentation and line breaks so that each document is easy to read, especially when there are multiple entries.

11, In this project, we use PostgreSQL for the User & Order Services and MongoDB for the Product Service. Why use two different databases in the Simple E-Commerce Microservices Project?

Ans -

In this project, we use PostgreSQL for the User and Order Services because these services require structured, relational data with strict consistency, such as user accounts, order IDs, and relationships between users and their orders, which are naturally suited to a relational database. MongoDB is used for the Product Service because product data is more flexible and document-oriented, with varying attributes like name, price, description, and category, which can change frequently. Using two different databases demonstrates polyglot persistence, where each service uses the database best suited to its data model and access patterns, improving scalability, flexibility, and maintainability in a microservices architecture.

12, Screenshot the output when shutting down all containers:

Ans -

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices % docker compose down
[+] Running 7/7
✔ Container assignment_microservices-order-service-1      Re...      1.3s
✔ Container assignment_microservices-user-service-1       Rem...     1.4s
✔ Container assignment_microservices-product-service-1    Removed     1.4s
✔ Container assignment_microservices-order-db-1           Removed     0.2s
✔ Container assignment_microservices-product-db-1         Remov...    0.3s
✔ Container assignment_microservices-user-db-1            Removed     0.2s
✔ Network assignment_microservices_default                Removed     0.2s
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3648 assignment_microservices %
```

13. In your own words, what accomplishments have you made through this project assignment?

Ans -

Through this Simple E-Commerce Microservices project, I have gained practical experience in designing and implementing a microservices architecture with multiple technologies. I successfully built and connected three independent services—User, Product, and Order—using Flask, Node.js, PostgreSQL, and MongoDB, demonstrating how different services communicate via REST APIs. I learned to containerize each service using Docker and manage them collectively with Docker Compose, ensuring they run consistently across environments. Additionally, I implemented database operations, including creating, retrieving, and deleting data, and practiced debugging and inspecting containers, which strengthened my understanding of database management and service orchestration in a distributed system. Overall, this project enhanced my skills in full-stack development, database integration, and microservices deployment, while reinforcing best practices for modular and maintainable system design.

14, How much time you spent on this assignment?

Ans - 60 minutes